## Unit – 5

### List in Python

A list is a collection of elements used to store multiple values in a single variable. A list is ordered, allows duplicate values and is mutable (its elements can be changed after creation).

**Syntax:**

list_name = [element1, element2, element3]

**Example:**

numbers = [10, 20, 30, 40, 50]

print(numbers)

**Output:**

[10, 20, 30, 40, 50]

### Basic Operations on List

#### 1. Accessing Elements

Elements of a list can be accessed using their index. Indexing starts from 0.

numbers = [10, 20, 30, 40, 50]

print(numbers[0])

print(numbers[2])

print(numbers[-1])

**Output:**

10

30

50

### numbers[-1] in List

In Python, negative indexing is used to access elements from the end of a list.

- -1 represents the last element.

- -2 represents the second-last element.
- -3 represents the third-last element.

**Example:**

```
numbers = [10, 20, 30, 40, 50]

print(numbers[-1])

print(numbers[-2])

print(numbers[-3])
```

**Output:**

```
50

40

30
```

**Explanation:** The index -1 accesses the last element, which is 50.

**2. Adding Elements**

Python provides the following methods to add elements to a list.

| Method | Description |
|---|---|
| append() | Adds an element at the end. |
| insert() | Adds an element at a specified position. |
| extend() | Adds multiple elements from another iterable. |

**Example:**

```
numbers = [10, 20, 30]

numbers.append(40)

print(numbers)

numbers.insert(1, 15)
```

```
print(numbers)

numbers.extend([50, 60])

print(numbers)
```

**Output:**

```
[10, 20, 30, 40]

[10, 15, 20, 30, 40]

[10, 15, 20, 30, 40, 50, 60]
```

**What is Iterable in extend() in List?**

An iterable is an object whose elements can be accessed one by one, such as a list, tuple, string or set.

The extend() method adds all elements of an iterable to the end of an existing list.

**Syntax:**

```
list.extend(iterable)
```

**Example 1: Using another List**

```
numbers = [10, 20, 30]

numbers.extend([40, 50])

print(numbers)
```

**Output:**

```
[10, 20, 30, 40, 50]
```

**Example 2: Using a String**

```
letters = ['A', 'B']

letters.extend("CD")

print(letters)
```

**Output:**

```
['A', 'B', 'C', 'D']
```

### 3. Removing Elements

| Method | Description |
|---|---|
| remove() | Removes the specified element. |
| pop() | Removes and returns an element at a specified index. |
| clear() | Removes all elements from the list. |

**Example:**

```
numbers = [10, 20, 30, 40]

numbers.remove(20)

print(numbers)

numbers.pop()

print(numbers)

numbers.clear()

print(numbers)
```

**Output:**

```
[10, 30, 40]

[10, 30]

[]
```

The **pop()** method removes an element from a list and returns the removed element.

It can remove an element using its index.

**Syntax:**

```
list.pop(index)
```

- If no index is provided, it removes the last element.
- If an index is provided, it removes the element at that position.
- It returns the removed element.

**Example 1: Without Index**

```
numbers = [10, 20, 30, 40]

x = numbers.pop()

print("Removed element:", x)

print("Updated list:", numbers)
```

**Output:**

Removed element: 40

Updated list: [10, 20, 30]

**Explanation:** Since no index is given, pop() removes the last element, 40, and stores it in variable x.

**Example 2: With Index**

```
numbers = [10, 20, 30, 40]

x = numbers.pop(1)

print("Removed element:", x)

print("Updated list:", numbers)
```

**Output:**

Removed element: 20

Updated list: [10, 30, 40]

**Explanation:** Index 1 represents the second element, so 20 is removed and returned.

**4. Updating Elements**

List elements can be modified by assigning a new value to a particular index.

**Example:**

```
numbers = [10, 20, 30]

numbers[1] = 50

print(numbers)
```

**Output:**

[10, 50, 30]

**5. Slicing a List**

Slicing is used to access a particular portion of a list.

**Syntax:**

list[start : stop : step]

There are three parts:

1). **Start:** The index from which slicing begins. It is included in the result.

2). **Stop:** The index where slicing ends. It is not included in the result.

3). **Step:** Specifies how many positions to move forward or backward between selected elements.

**Example without Step:**

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])

print(numbers[:3])

print(numbers[2:])
```

**Output:**

[20, 30, 40]

[10, 20, 30]

[30, 40, 50]

**Example with Step:**

```
numbers = [10, 20, 30, 40, 50, 60, 70]

print(numbers[1:5:1])

print(numbers[1:5:2])

print(numbers[::2])
```

```
print(numbers[::-1])
```

**Output:**

```
[20, 30, 40, 50]

[20, 40]

[10, 30, 50, 70]

[70, 60, 50, 40, 30, 20, 10]
```

**Explanation:**

- numbers[1:5:1]: Starts at index 1, stops before index 5, and moves one position at a time.
- numbers[1:5:2]: Starts at index 1 and moves two positions at a time.
- numbers[::2]: Selects every second element from the beginning to the end.
- numbers[::-1]: Reverses the list by moving backward.

Important: If start or stop is omitted, Python uses the beginning or end of the list as appropriate.

**6. Other Common List Functions**

| Function | Description |
|---|---|
| len() | Returns the number of elements. |
| sort() | Sorts elements in ascending order by default. |
| reverse() | Reverses the order of elements. |
| index() | Returns the index of an element. |
| count() | Returns the number of occurrences of an element. |

**Example:**

```
numbers = [30, 10, 20, 10]

print(len(numbers))
```

```
print(numbers.count(10))

print(numbers.index(20))

numbers.sort()

print(numbers)

numbers.reverse()

print(numbers)
```

**Output:**

4

2

2

[10, 10, 20, 30]

[30, 20, 10, 10]

**Tuple in Python**

A tuple is an ordered collection of elements used to store multiple values in a single variable. A tuple allows duplicate values but is immutable, which means its elements cannot be changed after creation.

**Syntax:**

```
tuple_name = (element1, element2, element3)
```

**Example:**

```
numbers = (10, 20, 30, 40, 50)

print(numbers)
```

**Output:** (10, 20, 30, 40, 50)

**Basic Operations on Tuple**

**1. Accessing Elements**

Tuple elements can be accessed using their index. Indexing starts from 0.

**Example:**

```
numbers = (10, 20, 30, 40, 50)

print(numbers[0])

print(numbers[2])

print(numbers[-1])
```

**Output:**

```
10

30

50
```

**2. Slicing a Tuple**

Slicing is used to access a specific portion of a tuple.

**Example:**

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])

print(numbers[:3])

print(numbers[2:])
```

**Output:**

```
(20, 30, 40)

(10, 20, 30)

(30, 40, 50)
```

**3. Concatenation**

Concatenation is used to combine two or more tuples using the + operator.

**Example:**

```
a = (10, 20, 30)
```

```
b = (40, 50, 60)

c = a + b

print(c)
```

**Output:**

```
(10, 20, 30, 40, 50, 60)
```

**4. Repetition**

The * operator is used to repeat the elements of a tuple.

**Example:**

```
numbers = (10, 20)

print(numbers * 3)
```

**Output:**

```
(10, 20, 10, 20, 10, 20)
```

**5. Tuple Methods**

| Method/Function | Description |
|---|---|
| len() | Returns the number of elements. |
| count() | Returns the number of occurrences of an element. |
| index() | Returns the index of an element. |
| max() | Returns the largest element. |
| min() | Returns the smallest element. |

**Example:**

```
numbers = (10, 20, 30, 20, 40)

print(len(numbers))

print(numbers.count(20))

print(numbers.index(30))
```

```
print(max(numbers))

print(min(numbers))
```

**Output:**

```
5

2

2

40

10
```

**6. Packing and Unpacking**

- Packing: Assigning multiple values to a single tuple.
- Unpacking: Assigning tuple elements to multiple variables.

Example:

```
student = ("Ankit", 20, "B.Tech")

name, age, course = student

print(name)

print(age)

print(course)
```

**Output:**

```
Ankit

20

B.Tech
```

**Note:** Tuple elements cannot be directly added, removed or modified after creation.

**3. Dictionary in Python**

A dictionary is a collection of data stored in the form of key-value pairs. Each key is used to access its corresponding value. Dictionaries are mutable and keys must be unique.

**Syntax:**

dictionary_name = {"key1": "value1", "key2": "value2"}

**Example:**

student = {"name": "Ankit", "age": 20, "course": "B.Tech"}

print(student)

Output:

{'name': 'Ankit', 'age': 20, 'course': 'B.Tech'}

**Basic Operations on Dictionary**

**1. Accessing Values**

Values can be accessed using their keys.

**Example:**

student = {"name": "Ankit", "age": 20, "course": "B.Tech"}

print(student["name"])

print(student["age"])

**Output:**

Ankit

20

The get() method can also be used to access values using keys.

**Example:**

print(student.get("course"))

Output:

B.Tech

**2. Adding Elements**

A new key-value pair can be added by assigning a value to a new key.

**Example:**

```
student = {"name": "Ankit", "age": 20}

student["course"] = "B.Tech"

print(student)
```

**Output:**

```
{'name': 'Ankit', 'age': 20, 'course': 'B.Tech'}
```

**3. Updating Elements**

The value of an existing key can be changed by assigning a new value.

**Example:**

```
student = {"name": "Ankit", "age": 20}

student["age"] = 21

print(student)
```

**Output:**

```
{'name': 'Ankit', 'age': 21}
```

The update() method can also update multiple key-value pairs.

**Example:**

```
student.update({"age": 22, "course": "B.Tech"})print(student)
```

**4. Removing Elements**

| Method | Description |
|---|---|
| pop() | Removes the specified key and returns its value. |
| popitem() | Removes and returns the last inserted key-value pair. |
| clear() | Removes all elements from the dictionary. |

**Example:**

```
student = {"name": "Ankit", "age": 20, "course": "B.Tech"}
```

```
student.pop("age")

print(student)

student.popitem()

print(student)

student.clear()

print(student)
```

**Output:**

```
{'name': 'Ankit', 'course': 'B.Tech'}

{'name': 'Ankit'}

{}
```

**5. Dictionary Methods**

| Method | Description |
|---|---|
| keys() | Returns all keys. |
| values() | Returns all values. |
| items() | Returns all key-value pairs. |
| get() | Returns the value of a specified key. |
| update() | Updates the dictionary. |
| len() | Returns the number of key-value pairs. |

**Example:**

```
student = {"name": "Ankit", "age": 20, "course": "B.Tech"}

print(student.keys())

print(student.values())

print(student.items())

print(len(student))
```

**Output:**

dict_keys(['name', 'age', 'course'])

dict_values(['Ankit', 20, 'B.Tech'])

dict_items([('name', 'Ankit'), ('age', 20), ('course', 'B.Tech')])

3

**6. Iterating Through a Dictionary**

A for loop can be used to access dictionary keys and values.

**Example:**

```
student = {"name": "Ankit", "age": 20, "course": "B.Tech"}

for key, value in student.items():

        print(key, ":", value)
```

**Output:**

name : Ankit

age : 20

course : B.Tech

**4. Set in Python**

A set is an unordered collection of unique elements. It does not allow duplicate values. Sets are useful for performing mathematical operations such as union, intersection and difference.

**Syntax:**

```
set_name = {element1, element2, element3}
```

**Example:**

```
numbers = {10, 20, 30, 40, 50}

print(numbers)
```

**Output:**

{10, 20, 30, 40, 50}

**Note:** Sets do not maintain a fixed order, so the output order may vary.

**Basic Operations on Set**

**1. Adding Elements**

The add() method adds a single element, whereas update() adds multiple elements.

**Example:**

```
numbers = {10, 20, 30}

numbers.add(40)

print(numbers)

numbers.update([50, 60])

print(numbers)
```

**Output:**

```
{10, 20, 30, 40}

{10, 20, 30, 40, 50, 60}
```

**2. Removing Elements**

| Method | Description |
|---|---|
| remove() | Removes the specified element. Gives an error if the element is absent. |
| discard() | Removes the specified element without an error if absent. |
| pop() | Removes and returns an arbitrary element. |
| clear() | Removes all elements. |

**Example:**

```
numbers = {10, 20, 30, 40}

numbers.remove(20)

print(numbers)

numbers.discard(50)
```

```
print(numbers)

numbers.pop()

print(numbers)

numbers.clear()

print(numbers)
```

**Note:** The element removed by pop() is not fixed because sets are unordered.

**3. Union Operation**

Union combines all unique elements of two sets.

**Operator:** |

Method: union()

**Example:**

```
A = {10, 20, 30}

B = {30, 40, 50}

print(A | B)
```

**Output:**

{10, 20, 30, 40, 50}

**4. Intersection Operation**

Intersection returns only the common elements present in both sets.

**Operator:** &

Method: intersection()

**Example:**

```
A = {10, 20, 30}

B = {30, 40, 50}

print(A & B)
```

**Output:**

{30}

**5. Difference Operation**

Difference returns elements that are present in the first set but not in the second set.

**Operator:** -

Method: difference()

**Example:**

A = {10, 20, 30}

B = {30, 40, 50}

print(A - B)

**Output:**

{10, 20}

**6. Symmetric Difference**

Symmetric difference returns elements that are present in either set but not in both sets.

**Operator:** ^

Method: symmetric_difference()

**Example:**

A = {10, 20, 30}

B = {30, 40, 50}

print(A ^ B)

**Output:**

{10, 20, 40, 50}

| **Method** | **Description** |
|---|---|
| len() | Returns the number of elements. |

| | |
|---|---|
| copy() | Creates a copy of the set. |
| issubset() | Checks whether all elements of one set are present in another set. |
| issuperset() | Checks whether a set contains all elements of another set. |
| isdisjoint() | Checks whether two sets have no common elements. |

**Example:**

A = {10, 20}

B = {10, 20, 30, 40}

print(A.issubset(B))

print(B.issuperset(A))

print(A.isdisjoint(B))

**Output:**

True

True

False

**5. Difference Between List, Tuple, Dictionary and Set**

| **Feature** | **List** | **Tuple** | **Dictionary** | **Set** |
|---|---|---|---|---|
| Symbol | [ ] | ( ) | {key: value} | { } |
| Order | Ordered | Ordered | Insertion ordered | Unordered |
| Duplicates | Allowed | Allowed | Keys are unique | Not allowed |
| Modification | Allowed | Not allowed | Allowed | Allowed |
| Access | Index | Index | Key | No indexing |
| Example | [1, 2, 3] | (1, 2, 3) | {"a": 1} | {1, 2, 3} |

**File Handling in Python**

File handling is used to create, read, write and modify files using Python. It allows us to store data permanently in files.

Python provides built-in functions such as open(), read(), write() and close() for file handling.

**1. Opening a File**

The open() function is used to open a file.

Syntax:

file = open("filename.txt", "mode")

**Common File Modes:**

| Mode | Description |
|---|---|
| r | Opens a file for reading. |
| w | Opens a file for writing. Creates a new file or overwrites existing content. |
| a | Opens a file for appending data at the end. |
| r+ | Opens a file for both reading and writing. |

**2. Writing to a Text File**

The write() function is used to write data into a text file.

**Program:**

file = open("student.txt", "w")

file.write("Welcome to Python Programming")

file.close()

**Explanation:**

**Line 1:**

file = open("student.txt", "w")

- open() opens the file named student.txt.
- "w" represents write mode.

- If the file does not exist, Python creates it.
- If the file already exists, its previous content is overwritten.
- The file object is stored in the variable file.

**Line 2:**

file.write("Welcome to Python Programming")

- write() writes the given text into the file.
- The text is stored in student.txt.
- It does not automatically add a new line.

**Line 3:**

file.close()

- close() closes the opened file.
- It ensures that the file is properly closed and pending changes are saved.

Content of student.txt:

Welcome to Python Programming

**3. Reading from a Text File**

The read() function is used to read the contents of a text file.

**Program:**

```
file = open("student.txt", "r")

data = file.read()

print(data)

file.close()
```

Output:

Welcome to Python Programming

**Explanation:**

**Line 1:**

file = open("student.txt", "r")

- Opens the file student.txt.
- "r" represents read mode.
- The file must already exist; otherwise, Python raises a FileNotFoundError.

**Line 2:**

data = file.read()

- read() reads the contents of the file.
- The complete content is returned as a string.
- The content is stored in the variable data.

**Line 3:**

print(data)

- Displays the content stored in the variable data.

**Line 4:**

file.close()

Closes the opened file.

**4. Reading a CSV File**

CSV stands for Comma-Separated Values. It is a file format used to store tabular data in rows and columns.

Python provides the built-in csv module to read and write CSV files.

Example of student.csv:

Name,Age,Course

Ankit,20,B.Tech

Rahul,21,BCA

**Program to Read a CSV File:**

```
import csv

file = open("student.csv", "r")

reader = csv.reader(file)

for row in reader:

        print(row)

file.close()
```

**Output:**

```
['Name', 'Age', 'Course']

['Ankit', '20', 'B.Tech']

['Rahul', '21', 'BCA']
```

**Explanation:**

**Line 1:**

```
import csv
```

- Imports the built-in csv module.
- This module provides functions to work with CSV files.

**Line 2:**

```
file = open("student.csv", "r")
```

- Opens the file named student.csv.
- "r" represents read mode.
- The file object is stored in the variable file.

**Line 3:**

```
reader = csv.reader(file)
```

- csv.reader() reads the CSV file.

- It separates each row into individual values using commas.
- The result is stored in the variable reader.

**Line 4:**

for row in reader:

- Uses a for loop to access each row of the CSV file one by one.
- Each row is returned as a list.

**Line 5:**

print(row)

- Displays each row of the CSV file.
- Each row is printed as a list of strings.

**Line 6:**

file.close()

Closes the CSV file after reading.

**Important File Handling Functions**

| **Function** | **Description** |
|---|---|
| open() | Opens a file. |
| read() | Reads the complete file content. |
| readline() | Reads one line from a file. |
| readlines() | Reads all lines and returns them as a list. |
| write() | Writes data into a file. |
| writelines() | Writes multiple strings into a file. |
| close() | Closes the file. |

**Introduction to NumPy**

NumPy (Numerical Python) is a Python library used for numerical calculations. It provides arrays and mathematical functions for efficient data processing.

**1. Creating a NumPy Array**

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])

print(a)
```

**Output:**

```
[10 20 30 40 50]
```

**Explanation:**

- import numpy as np imports the NumPy library.
- np.array() creates a NumPy array.
- Variable a stores the array elements.

**2. Calculating Mean**

The mean() function calculates the average of array elements.

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])

print(np.mean(a))
```

**Output:**

```
30.0
```

Explanation: Mean = Sum of elements / Number of elements.

**3. Simple Arithmetic Operations**

NumPy allows arithmetic operations directly on arrays.

```
import numpy as np

a = np.array([10, 20, 30])
```

```
print(a + 5)

print(a * 2)

print(a - 5)

print(a / 2)
```

**Output:**

```
[15 25 35]

[20 40 60]

[ 5 15 25]

[ 5. 10. 15.]
```

**Explanation:**

- a + 5: Adds 5 to each element.
- a * 2: Multiplies each element by 2.
- a - 5: Subtracts 5 from each element.
- a / 2: Divides each element by 2.

| Function | Description |
|---|---|
| np.array() | Creates an array. |
| np.mean() | Calculates the mean. |
| np.sum() | Calculates the sum. |
| np.max() | Returns the maximum value. |
| np.min() | Returns the minimum value. |

**Introduction to Matplotlib**

Matplotlib is a Python library used for data visualization. It is used to represent data in graphical form, such as line graphs, bar charts, pie charts and histograms.

It makes data easier to understand by displaying numerical information visually.

**Plotting a Simple Line Graph**

A line graph is used to represent data points connected by straight lines. It is useful for showing changes or trends in data.

**Python Program:**

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]

y = [10, 20, 15, 30, 25]

plt.plot(x, y)

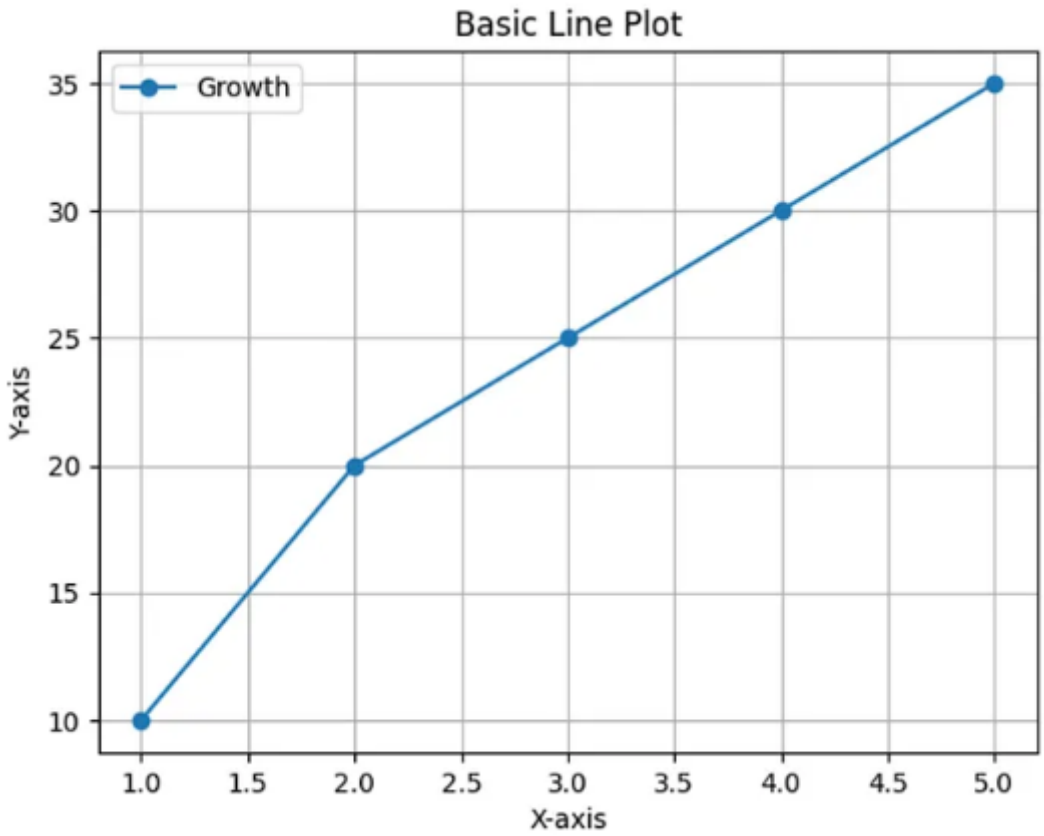
plt.title("Simple Line Graph")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")

plt.show()
```

**Output**



**Explanation of Program (Line by Line)**

**Line 1:**

import matplotlib.pyplot as plt

- import is used to include a library in Python.
- matplotlib.pyplot is a module of Matplotlib used for plotting graphs.
- as plt gives a short name to the module.
- We use plt instead of writing matplotlib.pyplot repeatedly.

**Line 2:**

x = [1, 2, 3, 4, 5]

- Creates a list named x.
- It contains values for the X-axis (horizontal axis).
- These values represent the positions of data points on the horizontal axis.

**Line 3:**

y = [10, 20, 15, 30, 25]

- Creates a list named y.
- It contains values for the Y-axis (vertical axis).
- These values represent the corresponding data points on the vertical axis.

**Line 4:**

plt.plot(x, y)

- plot() is used to create a line graph.
- It takes two lists: x and y.
- It connects the corresponding data points using straight lines.

The points created are:

| X-axis | Y-axis | Coordinate |
|---|---|---|
| | | |

| | | |
|---|---|---|
| 1 | 10 | (1, 10) |
| 2 | 20 | (2, 20) |
| 3 | 15 | (3, 15) |
| 4 | 30 | (4, 30) |
| 5 | 25 | (5, 25) |

**Line 5:**

plt.title("Simple Line Graph")

- title() is used to give a title to the graph.
- Here, the title displayed is "Simple Line Graph".

**Line 6:**

plt.xlabel("X-axis")

- xlabel() is used to provide a label to the horizontal axis.
- Here, the horizontal axis is named "X-axis".

**Line 7:**

plt.ylabel("Y-axis")

- ylabel() is used to provide a label to the vertical axis.
- Here, the vertical axis is named "Y-axis".

**Line 8:**

plt.show()

- show() displays the created graph on the screen.
- Without this statement, the graph may not appear in some Python environments.

**Important Matplotlib Functions**

| **Function** | **Description** |
|---|---|
| plt.plot() | Creates a line graph. |

| | |
|---|---|
| plt.title() | Adds a title to the graph. |
| plt.xlabel() | Labels the X-axis. |
| plt.ylabel() | Labels the Y-axis. |
| plt.show() | Displays the graph. |
| plt.grid() | Displays grid lines. |
| plt.legend() | Displays a legend. |

**Advantages of Matplotlib**

**1).** It makes data easier to understand through graphical representation.

**2).** It supports different types of graphs and charts.

**3).** It is useful in data analysis and scientific computing.

**4).** It provides customization options for titles, labels, colours and styles.

**5).** It works with NumPy and Pandas libraries.

**Mini-Demo: IoT Sensor Simulation Using Python**

IoT (Internet of Things) refers to a network of physical devices that collect and exchange data using the internet. Sensors are used to collect information such as temperature, humidity and pressure. The following **program** simulates a temperature sensor that generates random temperature readings.

```
import random
import time
for i in range(5):
    temperature = random.randint(20, 40)
    print("Temperature Sensor Reading:", temperature, "°C")
    if temperature > 35:
        print("Warning: High Temperature!")
    time.sleep(1)
```

**Output**

Temperature Sensor Reading: 28 °C

Temperature Sensor Reading: 32 °C

Temperature Sensor Reading: 38 °C

Warning: High Temperature!

Temperature Sensor Reading: 25 °C

Temperature Sensor Reading: 36 °C

Warning: High Temperature!

**Explanation**

**Line 1:**

import random

Imports the random module to generate random temperature values.

**Line 2:**

import time

Imports the time module to create a delay between sensor readings.

**Line 3:**

for i in range(5):

Runs the loop five times to simulate five sensor readings.

**Line 4:**

temperature = random.randint(20, 40)

Generates a random integer between 20°C and 40°C and stores it in the variable temperature.

**Line 5:**

print("Temperature Sensor Reading:", temperature, "°C")

Displays the temperature reading on the screen.

**Line 6:**

if temperature > 35:

Checks whether the temperature is greater than 35°C.

**Line 7:**

print("Warning: High Temperature!")

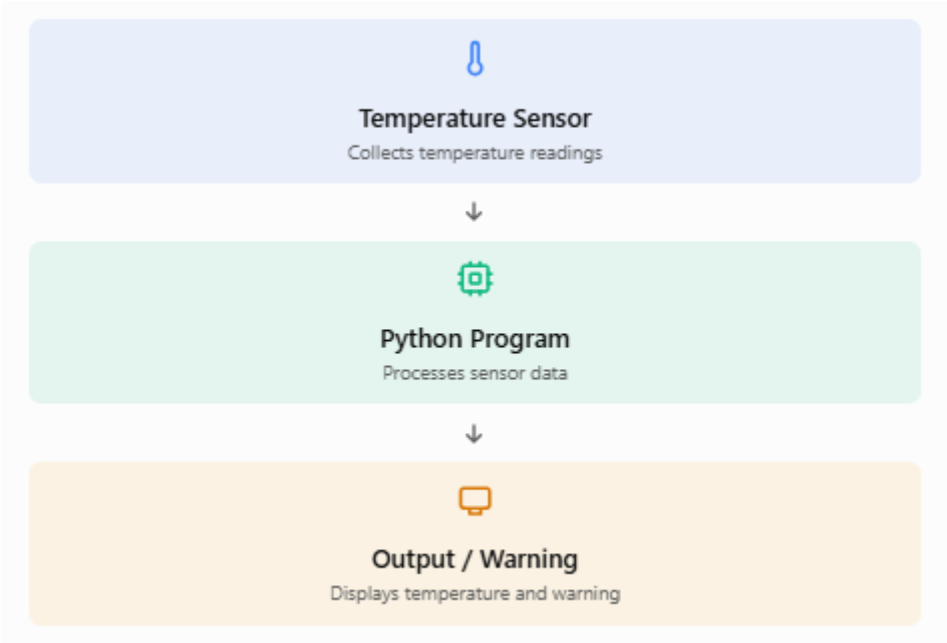
Displays a warning if the temperature exceeds 35°C.

**Line 8:**

time.sleep(1)

Pauses the program for one second before taking the next reading.

**Working of IoT Sensor Simulation**




**Applications:**

1). Smzart homes.

2). Weather monitoring.

3). Industrial temperature monitoring.

4). Smart agriculture.